
System Design Concepts

Credit: @sarasyntax

Introduction

If you want to level up from a junior developer to a senior engineer, or land a high-paying job at a big tech company, you need to learn system design. The hard part is knowing where to start. System design feels broad because it touches networking, data, APIs, scaling, reliability, and architecture choices all at once.

A strong foundation comes from understanding the core patterns that appear across most software systems. Once those patterns feel familiar, larger designs become much easier to reason about.

1. Client-Server Architecture

Almost every web application is built on the simple but powerful concept of client-server architecture. On one side you have a client, such as a web browser, a mobile app, or any frontend application. On the other side you have a server, a machine that runs continuously and waits for incoming requests.

The client sends a request to store, retrieve, or modify data. The server receives the request, processes it, performs the required logic, and sends back a response. That basic request-response loop is the foundation of most modern applications.

This matters because system design usually starts by asking where responsibilities live. The client mainly handles presentation and user interaction, while the server is responsible for data access, business logic, and coordination with databases or other services.

2. IP Addresses and Domain Names

A client cannot talk to a server unless it knows where that server is. On the internet, machines identify each other using IP addresses. You can think of an IP address like a phone number for a server.

Every publicly deployed server has an address that other machines can use to reach it. But users do not want to memorize raw numeric addresses. That is why systems expose human-friendly names such as website domains instead.

A domain name gives a stable public identity to a service, even if the actual server infrastructure changes underneath it. That separation is valuable because it lets teams move services, rotate infrastructure, or fail over traffic without changing the public entry point.

3. DNS

DNS, or the Domain Name System, maps human-friendly domain names to their corresponding IP addresses. When you type a website name into a browser, the client first asks DNS which IP address that name points to.

Once DNS responds with the IP, the browser can establish a connection and send the request to the right server. Without DNS, using the internet at scale would be much more painful because every destination would require remembering a raw IP address.

DNS is more than a lookup table. In practice, it also affects latency, failover, global routing, and caching behavior because DNS responses can be cached and updated strategically across networks.

4. Proxy and Reverse Proxy

When you visit a website, your request does not always go straight to the final application server. Sometimes it passes through a proxy or a reverse proxy first.

A proxy server acts as a middleman between a client and the internet. It forwards requests to the target server, receives the response, and returns it to the client. One common benefit is privacy because it can hide the client IP address.

A reverse proxy works in the opposite direction. It sits in front of backend servers and intercepts incoming client requests. It then forwards them to the correct internal server based on routing rules. Reverse proxies are common because they can handle TLS termination, request routing, caching, compression, and protection of backend

services in one place.

5. Latency

Whenever a client communicates with a server, there is some delay. One of the biggest causes of delay is physical distance. If a server is in New York and a user is in India, the request and response must travel across a large geographic distance.

That round-trip delay is called latency. High latency makes applications feel slow, even when the application logic itself is correct.

One of the most common ways to reduce latency is to deploy services across multiple regions or data centers so that users can connect to the nearest infrastructure instead of always communicating with one distant location.

6. HTTP

Once a connection is made, the client and server need rules for communication. HTTP is the protocol used for most web traffic. That is why URLs often start with HTTP or HTTPS.

An HTTP request contains information such as the method, headers, and sometimes a request body. A server processes that request and sends back an HTTP response, which includes a status code, headers, and response data or an error.

HTTP is stateless by default. Each request stands on its own unless extra mechanisms such as cookies, sessions, or tokens are introduced to preserve context.

7. HTTPS, SSL, and TLS

Plain HTTP has a major weakness: data is sent in readable form. That means a malicious party intercepting the traffic may be able to see or alter the content.

HTTPS solves that by encrypting data in transit using TLS, historically discussed alongside SSL. With HTTPS, the client and server establish a secure channel before exchanging sensitive data.

For modern systems, HTTPS is the baseline. It protects credentials, personal information, tokens, and business data while also reducing the risk of tampering in transit.

8. APIs

Clients and servers do not just exchange random HTTP messages. They communicate through APIs, or application programming interfaces, which define how requests should be structured and how responses should be interpreted.

An API acts like a contract. The client sends a request, the server processes it, talks to databases or other services if needed, and returns a structured response, often in JSON or XML.

Strong API design improves maintainability because frontend and backend teams can evolve independently as long as they preserve the contract and expected behavior.

9. REST

REST is one of the most widely used API styles. It treats data and capabilities as resources such as users, products, or orders, and it uses standard HTTP methods such as GET, POST, PUT, and DELETE.

REST is attractive because it is simple, familiar, and works naturally with web infrastructure. It is also easier to cache than many other API styles, which can help with performance.

Its weakness appears when clients need complex or highly customized data. In those situations, REST endpoints may return more information than needed or require multiple requests to gather related data efficiently.

10. GraphQL

GraphQL was introduced to let clients request exactly the fields they need. Instead of calling multiple endpoints and combining responses on the client side, a client can often fetch everything through a single query.

This is especially useful for complex user interfaces where different screens want slightly different slices of data. It reduces over-fetching and under-fetching.

The tradeoff is that GraphQL shifts more complexity to the server. Query validation, resolver efficiency, authorization, and protection against expensive nested queries become important very quickly.

11. Databases

Applications usually need to store and retrieve information reliably, which is why databases are foundational in system design. Keeping everything in process memory or in loose files is not enough for serious systems.

A database helps keep data durable, queryable, secure, and manageable over time. The application server receives the user request, talks to the database, and then returns the result to the client.

When discussing a system, always ask what the core entities are, which queries are common, how much data will exist, and what consistency guarantees the product actually needs.

12. SQL and NoSQL

In system design, one of the most common database choices is between SQL and NoSQL. SQL databases store data in structured tables with predefined schemas and are often chosen for transactional systems that need strong consistency and relational integrity.

NoSQL systems are designed for high scalability, flexible schemas, and specialized access patterns. They may use key-value, document, graph, or wide-column models depending on the problem.

The choice depends on the workload. If you need strong relationships and transactional guarantees, SQL is often a better fit. If you need scale, flexibility, or distributed access patterns, NoSQL may be better. Many production systems use both where appropriate.

13. Vertical Scaling

As traffic grows, one fast response is to make the existing server bigger by adding more CPU, more RAM, or more storage. This is called vertical scaling or scaling up.

Vertical scaling is easy to understand and often easy to implement in the short term. But it has clear limits. Hardware cannot be upgraded forever, larger machines become much more expensive, and a single powerful server can still be a single point of failure.

That is why vertical scaling is often a temporary step rather than a complete long-term architecture strategy.

14. Horizontal Scaling

A more scalable pattern is horizontal scaling, also called scaling out. Instead of making one server stronger, you add more servers and distribute the workload among them.

This increases total capacity and also improves resilience because if one machine fails, other machines can keep serving traffic.

The challenge is coordination. Once you have multiple machines, you need to route traffic, manage shared state carefully, and reason about distributed failure scenarios in a disciplined way.

15. Load Balancers

When many backend servers exist, clients should not manually choose which one to call. A load balancer sits between clients and servers and distributes requests across the available machines.

This helps spread traffic, avoid overloaded nodes, and redirect traffic away from unhealthy instances. It is a central pattern in scalable web systems.

Common strategies include round robin, least connections, and IP-based hashing. In good system design answers, it is useful to mention health checks, failover behavior, and the effect on session handling.

16. Indexing

As databases grow, query speed becomes critical. Indexing is one of the fastest ways to improve read performance. It works like the index at the back of a book, allowing the database to jump closer to the data without scanning the full table.

Indexes are usually built on columns that are frequently searched, sorted, or used in joins. This can dramatically reduce lookup time.

The downside is that writes become more expensive because indexes must be updated whenever indexed data changes. That is why indexes should be added thoughtfully rather than applied everywhere.

17. Replication

If one database server cannot handle all reads efficiently, one common technique is replication. A primary database handles writes, while one or more replicas receive copied data and serve read traffic.

This improves read scalability and can also improve availability because a replica may be promoted if the primary fails.

Replication introduces important tradeoffs. Replicas can lag behind the primary, which means some reads may be slightly stale depending on the replication model and failure conditions in the system.

18. Sharding

When a dataset becomes too large for one database server, replication alone is not enough because it helps mostly with reads. Sharding addresses this by splitting data across multiple servers.

Each shard stores only a subset of the total data, often chosen by a shard key such as user ID. This spreads both storage and query load across machines.

Sharding can dramatically improve scale, but it makes the architecture harder. Picking the wrong shard key can create hot partitions, uneven load, and difficult cross-shard queries.

19. Vertical Partitioning

Sometimes the problem is not the number of rows but the number of columns in a very wide table. Vertical partitioning solves this by splitting a table by columns instead of rows.

For example, profile information, billing information, and login history may be separated if different queries use them differently. This reduces unnecessary I/O and can speed up common queries.

The tradeoff is that some use cases later need to combine data from multiple partitions, which can reintroduce complexity through extra joins or coordination.

20. Caching

Even a well-tuned database is slower than memory. Caching improves performance by storing frequently accessed data in a faster layer, often in memory.

A very common approach is cache-aside. The application checks the cache first. If the data exists there, it returns it immediately. If not, the application fetches the data from the database, stores it in cache, and returns it.

Caching sounds simple, but invalidation is the hard part. You need to think about TTL values, stale reads, eviction, and how to refresh or remove outdated entries safely and predictably.

21. Denormalization

Relational systems often normalize data to reduce duplication, but normalization can lead to many joins during reads. As datasets grow, those joins can become expensive.

Denormalization improves read speed by storing related data together, even if that means duplicating some information. For example, instead of joining users and orders every time, a system may store a read-optimized combined representation.

This is common in read-heavy systems. The tradeoff is that writes become more complex because duplicated data must be kept in sync across multiple places.

22. CAP Theorem

As systems spread across multiple machines and data centers, they become distributed systems. CAP theorem is one of the classic ideas for reasoning about them.

It states that during a network partition, a distributed system cannot fully guarantee both strong consistency and full availability at the same time. Since partitions are unavoidable in real distributed systems, tradeoffs must be made.

This does not mean every design is only one thing forever. It means engineers need to decide where the system should prefer consistent behavior and where it can tolerate temporary inconsistency in order to remain available.

23. Blob Storage

Modern applications do not store only text records. They also handle images, videos, PDFs, backups, and other large files. Traditional relational databases are usually not the best place to store those large binary objects.

Blob storage services such as Amazon S3 are optimized for this problem. Files are stored as objects in buckets, and they can be retrieved using identifiers or URLs.

This separation is powerful because metadata can stay in a database while the heavy file content lives in scalable object storage that is easier to serve and replicate.

24. CDN

Serving large static assets or video files directly from one distant origin server can create slow experiences for users around the world. A content delivery network, or CDN, solves this by placing content on geographically distributed edge servers.

Users then fetch content from a nearby location, which reduces latency and buffering. CDNs are especially valuable for images, JavaScript, CSS, downloads, and media streaming.

They also reduce origin load because repeated requests are answered from the edge instead of reaching the central server every time, which improves overall efficiency.

25. WebSockets

HTTP uses a request-response model, which is fine for many applications but inefficient for real-time communication. If a client needs updates every second, repeated polling creates waste and delay.

WebSockets solve that by creating a long-lived, persistent connection between client and server. Once the connection is established, both sides can send messages at any time.

This is useful for live chat, stock dashboards, multiplayer games, notifications, and collaborative tools. At scale, it also raises new concerns such as connection management, fan-out, and message delivery across multiple servers reliably.

26. Webhooks

WebSockets help clients and servers communicate in real time, but sometimes one server needs to notify another server about an event. That is where webhooks come in.

A webhook is essentially an event-driven HTTP callback. Your system registers a URL, and when an event occurs, the provider sends an HTTP request to that URL with the event details.

This avoids wasteful polling and makes integrations more responsive. In production, it is also important to think about retries, signatures, authentication, and idempotency for safe processing.

27. Monoliths and Microservices

Many applications begin as monoliths, where all features live inside one codebase and one deployment unit. This is often productive in the beginning because everything is simple to run and debug.

As the system grows, the monolith may become harder to scale, deploy, or manage. One response is to break it into microservices, where each service owns one main responsibility and can be deployed independently.

Microservices improve autonomy and scaling flexibility, but they also introduce network communication, operational overhead, distributed tracing, and many more failure modes that teams must handle well.

28. Message Queues

When multiple services need to communicate, synchronous API calls are not always ideal. Message queues allow one service to place work into a queue while another service processes it asynchronously.

This decouples producers from consumers and helps systems absorb traffic spikes. It is especially useful for notifications, media processing, analytics pipelines, and background jobs.

A complete design usually also considers retries, dead-letter queues, ordering guarantees, and which delivery semantics are acceptable for the business case.

29. Rate Limiting

Public APIs and user-facing services need protection from abuse, bots, accidental loops, and traffic spikes. Rate limiting controls how many requests a client can make in a given time period.

A system might allow 100 requests per minute per user or per IP. If the limit is exceeded, extra requests are blocked or delayed. This protects the service and preserves capacity for legitimate traffic.

Common strategies include fixed window, sliding window, and token bucket. In real systems, rate limiting often lives at the API gateway or edge layer where enforcement is easier to centralize.

30. API Gateways

An API gateway acts as a single entry point for clients in front of multiple backend services. Instead of exposing every service directly, the gateway receives requests and routes them to the right internal destination.

Gateways often handle authentication, rate limiting, logging, monitoring, and request routing in one place. This reduces duplicated edge logic across services.

The gateway becomes a critical shared component, so reliability, scaling, and observability for the gateway itself are extremely important.

31. Idempotency

In distributed systems, retries are common. A request might be repeated because of a timeout, a user refresh, or a client retry policy. Without protection, the same logical action could be processed twice.

Idempotency ensures that repeating the same request produces the same result as if it happened only once. This is especially important for actions such as payments, order creation, and booking workflows.

A common pattern is to attach a unique idempotency key to a request, store the result of the first successful execution, and return that stored result if the same logical request arrives again later.